RExercise#3_Porras

Renz Benedict Porras

2026-10-14

Part A — Creating Vectors: c(), seq(), rep()

```
# TODO: Create a numeric vector called scores containing: 88, 92, 79, 95, 84
# Then create a character vector called subjects containing:
# "Math", "Science", "English", "History", "PE"

scores <- c(88, 92, 79, 95, 84)
subjects <- c("Math", "Science", "English", "History", "PE")

scores
```

```
## [1] 88 92 79 95 84
```

```
subjects
```

```
## [1] "Math"    "Science" "English" "History" "PE"
```

```
# TODO: Use seq() to create a vector called odds containing odd numbers
# from 1 to 19 (hint: use the "by" argument)

# TODO: Use seq() with length.out to create a vector of exactly 5 evenly
# spaced numbers between 0 and 1

odds <- seq(from = 1, to = 19, by = 2)
odds
```

```
##  [1]  1  3  5  7  9 11 13 15 17 19
```

```
seq(from = 0, to = 1, length.out = 5)
```

```
## [1] 0.00 0.25 0.50 0.75 1.00
```

```
# TODO: Use rep() to create a vector that repeats the value "A" 4 times

rep("A", times = 4)
```

```
## [1] "A" "A" "A" "A"
```

```
# TODO: Use rep() with the "each" argument to turn c("A", "B") into
# "A" "A" "B" "B" (each element repeated twice, in order)

rep(c("A", "B"), each = 2)
```

```
## [1] "A" "A" "B" "B"
```

```
# TODO: Use rep() with the "times" argument to turn c("A", "B") into
# "A" "B" "A" "B" (the whole sequence repeated twice)
```

```r
rep(c("A", "B"), times = 2)
```

```
## [1] "A" "B" "A" "B"
```

Question #1:

```r
# Set up a vector named age, consisting of 15, 28, 32, 36, 27, 18, 52, 39, 42, 29, 35, 31, 27, 22, 37, 

age <- c(15, 28, 32, 36, 27, 18, 52, 39, 42, 29, 35, 31, 27, 22, 37, 34, 19, 20, 57, 49, 50, 37, 46, 25
```

```r
# a. How many data points?
length(age)
```

```
## [1] 34
```

```r
# b. Find the reciprocal of age?

1 / age
```

```
##  [1] 0.06666667 0.03571429 0.03125000 0.02777778 0.03703704 0.05555556
##  [7] 0.01923077 0.02564103 0.02380952 0.03448276 0.02857143 0.03225806
## [13] 0.03703704 0.04545455 0.02702703 0.02941176 0.05263158 0.05000000
## [19] 0.01754386 0.02040816 0.02000000 0.02702703 0.02173913 0.04000000
## [25] 0.05882353 0.08333333 0.02380952 0.01886792 0.02439024 0.01960784
## [31] 0.02857143 0.04166667 0.03030303 0.12500000
```

```r
# c. Assign also new_age <- c(age, 0, age) . What happens to new_age ?

new_age <- c(age, 0, age)
new_age
```

```
##  [1] 15 28 32 36 27 18 52 39 42 29 35 31 27 22 37 34 19 20 57 49 50 37 46 25 17
## [26] 12 42 53 41 51 35 24 33  8  0 15 28 32 36 27 18 52 39 42 29 35 31 27 22 37
## [51] 34 19 20 57 49 50 37 46 25 17 12 42 53 41 51 35 24 33  8
```

```r
# Answer: it combines age, 0, and age into one vector of length 69.
```

```r
# d: Find the minimum and the maximum value of age.

min(age)
```

```
## [1] 8
```

```r
max(age)
```

```
## [1] 57
```

Question #2:

```r
# rep(c("A", "B"), each = 2) and rep(c("A", "B"), times = 2) both produce a vector of length 4, but wit
# Explain in your own words the difference between each and times .

rep(c("A", "B"), each = 2)
```

```
## [1] "A" "A" "B" "B"
```

```r
rep(c("A", "B"), times = 2)
```

```
## [1] "A" "B" "A" "B"
```

```r
# Answer: each repeats every element by sequence, while times repeats the entire sequence or alternatel
```

Question #3:

```r
# Compare seq(1, 20, by = 2) with seq(1, 20, length.out = 5) . What does each argument ( by vs length.o

seq(1, 20, by = 2)
```

```
##  [1]  1  3  5  7  9 11 13 15 17 19
```

```r
seq(1, 20, length.out = 5)
```

```
## [1]  1.00  5.75 10.50 15.25 20.00
```

```r
# Answer: by sets the exact step size between numbers, length.out sets the exact total count of numbers
```

Question #4:

```r
# a. Generate a sequence for 20 to 60. Name the vector as vec_num

vec_num <- 20:60
vec_num
```

```
##  [1] 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44
## [26] 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60
```

```r
# b. Get the mean for vec_num .

mean(vec_num)
```

```
## [1] 40
```

```r
# c. Get the sum for vec_num .

sum(vec_num)
```

```
## [1] 1640
```

```r
# d: How many data points are there in vec_num ?

length(vec_num)
```

```
## [1] 41
```

Part B — Subsetting (Accessing) Vectors

```r
grades <- c(88, 92, 79, 95, 84)
names(grades) <- c("Math", "Science", "English", "History", "PE")

# TODO: Access the 3rd element of grades using positional indexing
# TODO: Access the 1st abd 4th element of grades using positional indexing
# TODO: Access "Science" and "PE" together using their names
```

```r
grades[3]
```

```
## English
##      79
```

```r
grades[c(1, 4)]
```

```
##    Math History
##      88      95
```

```r
grades[c("Science", "PE")]
```

```
## Science      PE
##      92      84
```

```r
# TODO: Return all elements of grades EXCEPT the 1st and 4th, using
# negative indexing, e.g. grades[-c(...)]

grades[-c(1, 4)]
```

```
## Science English      PE
##      92      79      84
```

```r
# TODO: Use a logical condition to return only the grades that are
# greater than or equal to 85 (hint: grades[grades >= 85])

grades[grades >= 85]
```

```
##    Math Science History
##      88      92      95
```

Question #5:

```r
# Negative indexing ( grades[-1] ) removes an element, while logical indexing ( grades[grades >= 85] ) 
# Describe a real situation where you would need negative indexing, and a different situation where log

grades[-1]
```

```
## Science English History      PE
##      92      79      95      84
```

```r
grades[grades >= 85]
```

```
##    Math Science History
##      88      92      95
```

```r
# Answer: Negative indexing is used when you know the exact positions to drop, such as removing an unwa

# Logical indexing is used when filtering data based on criteria without knowing their positions before
```

Question #6:

```r
# What happens if you run grades[10] (an index that doesn't exist)? Run it, observe the result, and exp

grades[10]
```

```
## <NA>
##   NA
```

```
# Answer: R returns NA instead of an error because vectors are treated as having undefined or missing v
```

Question #7:

```
# Print a vector with the integers between 1 and 100 that are not divisible by 3, 5 and 7 using filter
# Filter(function(i) { all(i %% c(3,5,7) != 0) }, seq(100))
# Write the R code and its output.

result <- Filter(function(i) { all(i %% c(3,5,7) != 0) }, seq(100))

result
```

```
##  [1]  1  2  4  8 11 13 16 17 19 22 23 26 29 31 32 34 37 38 41 43 44 46 47 52 53
## [26] 58 59 61 62 64 67 68 71 73 74 76 79 82 83 86 88 89 92 94 97
```

Part C — Vector Operations and append()

```
x <- c(1, 2, 3, 4)
y <- c(10, 20, 30, 40)

# TODO: Add x and y element-wise, and print the result
# TODO: Multiply every element of x by 2

x + y
```

```
## [1] 11 22 33 44
```

```
x * 2
```

```
## [1] 2 4 6 8
```

```
z <- c(1, 2, 3, 4, 5, 6)
short <- c(1, 0)

# TODO: Add z and short together and print the result. Run this even
# though the vectors are different lengths -- observe what R does.

z + short
```

```
## [1] 2 2 4 4 6 6
```

```
fruits <- c("apple", "banana", "cherry")

# TODO: Use append() to add "date" to the end of fruits
# TODO: Use append() with the "after" argument to insert "kiwi" between
# "apple" and "banana" (i.e., after position 1)

append(fruits, "date")
```

```
## [1] "apple"  "banana" "cherry" "date"
```

```
append(fruits, "kiwi", after = 1)
```

```
## [1] "apple"  "kiwi"   "banana" "cherry"
```

Question #8:

```
# In the vectors-recycling chunk, z has 6 elements and short has only 2. R still produced an answer ins
# Explain, in your own words, what R's "recycling rule" did to short to make the addition work.

z + short
```

```
## [1] 2 2 4 4 6 6
```

```
# Answer: R repeated (recycled) the shorter vector c(1, 0) three times to match the length of z (6 elem
```

Question #9:

```
# Now try adding z (length 6) to c(1, 0, 1) (length 3) instead. Does it work cleanly?

z + c(1, 0, 1)
```

```
## [1] 2 2 4 5 5 7
```

```
# Answer: Yes, it works cleanly without warnings because 6 is an exact multiple of 3.
```

```
# Then try adding z to c(1, 0, 1, 1) (length 4). Compare the warning (or lack of one) in each case, and

z + c(1, 0, 1, 1)
```

```
## Warning in z + c(1, 0, 1, 1): longer object length is not a multiple of shorter
## object length
```

```
## [1] 2 2 4 5 6 6
```

```
# Answer: Length 3 produces no warning, whereas length 4 produces a warning because 6 is not a multiple
```

Part D — Lists: Creating and Subsetting

```
# TODO: Create a list called student with these named elements:
# name = "Maria Santos", age = 20, grades = c(88, 92, 79),
# is_enrolled = TRUE

student <- list(
  name = "Maria Santos",
  age = 20,
  grades = c(88, 92, 79),
  is_enrolled = TRUE
)

student
```

```
## $name
## [1] "Maria Santos"
##
## $age
## [1] 20
##
## $grades
## [1] 88 92 79
##
```

```
## $is_enrolled
## [1] TRUE
```

```
# TODO: Access the "grades" element of student using single brackets
# student["grades"] -- then check its type with class()

student["grades"]
```

```
## $grades
## [1] 88 92 79
```

```
class(student["grades"])
```

```
## [1] "list"
```

```
# TODO: Access the "grades" element of student using double brackets
# student[["grades"]] -- then check its type with class()

student[["grades"]]
```

```
## [1] 88 92 79
```

```
class(student[["grades"]])
```

```
## [1] "numeric"
```

```
# TODO: Access the "name" element of student using the $ operator

student$name
```

```
## [1] "Maria Santos"
```

```
# TODO: From student, extract just the SECOND grade in the grades vector
# in a single expression, e.g. student[["grades"]][2] or student$grades[2]

student[["grades"]][2]
```

```
## [1] 92
```

Question #10:

```
# student["grades"] and student[["grades"]] look similar but return objects of different classes.
# Run class() on each and explain the difference in what single-bracket [ ] versus double-bracket [[ ]]

class(student["grades"])
```

```
## [1] "list"
```

```
class(student[["grades"]])
```

```
## [1] "numeric"
```

```
# Answer: Single bracket [ ] returns a sub-list containing the element, while double bracket [[ ]] extr
```

Question #11:

```r
# Why do you think R makes a distinction between [ ] and [[ ]] for lists, but a vector only needs [ ] ?

# Answer: Vectors hold simple, identical items, while lists hold complex, varied containers that requir
```

Part E — Critical Thinking Challenge

```r
inventory <- list(
  item = c("Notebook", "Pen", "Eraser"),
  price = c(25.50, 10.00, 5.00),
  in_stock = c(TRUE, TRUE, FALSE)
)

# TODO: Using what you learned in Parts B and D, print only the items
# (from inventory$item) that are currently in stock (in_stock == TRUE)

inventory$item[inventory$in_stock]
```

```
## [1] "Notebook" "Pen"
```

```r
# TODO: A new product "Ruler" priced at 15.00 and in stock (TRUE) needs
# to be added to inventory. Use append() to add "Ruler" to
# inventory$item, 15.00 to inventory$price, and TRUE to inventory$in_stock.
# (You will need three separate append() calls, one per vector.)

inventory$item <- append(inventory$item, "Ruler")
inventory$price <- append(inventory$price, 15.00)
inventory$in_stock <- append(inventory$in_stock, TRUE)

inventory
```

```
## $item
## [1] "Notebook" "Pen"      "Eraser"   "Ruler"
##
## $price
## [1] 25.5 10.0  5.0 15.0
##
## $in_stock
## [1]  TRUE  TRUE FALSE  TRUE
```

Question #12:

```r
# In challenge-1 , you combined logical indexing on one vector ( in_stock ) to filter a different vecto
# Explain, step by step, why inventory$item[inventory$in_stock] works even though item and in_stock are

# Answers:
# 1. inventory$in_stock extracts the logical vector c(TRUE, TRUE, FALSE).
# 2. inventory$item extracts the character vector c("Notebook", "Pen", "Eraser").
# 3. R uses positional alignment to keep elements where corresponding logical indices are TRUE (positio
```

Question #13:

```
# Suppose a classmate stores related data as three separate, unconnected vectors ( item_names , item_pr

# Answer: Storing data in separate vectors risks misaligning rows if one vector gets sorted or filtered
```

CONCLUSION

```
# In 3-4 sentences, summarize the key differences between vectors and lists in R, and explain when you

# Answer: All in all, vectors hold elements of the exact same type, making them perfect for fast math a
```